

CHOFYAI STUDIO

07 · Base de datos y persistencia

Documento	07-database.md
Versión analizada	0.5.1 (commit f840055)
Fecha del análisis	2026-08-28
Fuente Markdown	docs/system-documentation/07-database.md
Generado por	scripts/docs/build-pdf.mjs

Contenido

1. Conclusión previa: no existe motor de base de datos
 - 1.1. Evidencia de la búsqueda
2. Inventario de almacenes
 - 2.1. Resolución de la ruta de ``settings.json``
3. Diccionario de datos
 - 3.1. A1 — ``settings.json`` (`AppSettings``)
 - 3.2. A1b — Mapa ``tool_overrides``
 - 3.3. A2 — ``processes.json`` (mapa ``tool_id`` → PID)
 - 3.4. A3 — ``crash.log``
 - 3.5. A4/A5/A6 — Logs por herramienta
 - 3.6. A7 — Manifests de herramientas (`apps/*.yaml`` → ``RawManifest``)
 - 3.7. A8 — Workflows (`workflows/*.yaml``)
 - 3.8. A9 — ``marketplace/registry.yaml`` (`MarketplaceEntry``)
 - 3.9. A10 — Árbol de ``studio_home``
 - 3.10. A11 — ``localStorage`` del frontend
4. Relaciones entre entidades
 - 4.1. Diagrama entidad-relación
5. Reglas de integridad implementadas
 - 5.1. Unicidad del ``id`` de manifest
6. Modo de fallo por almacén
7. Mapa de módulo/función por almacén
8. Concurrencia y transaccionalidad
 - 8.1. Lo que sí está protegido
 - 8.2. Lo que NO está protegido
 - 8.3. Resumen de garantías
9. Datos personales y sensibles almacenados
 - 9.1. Hallazgo: ``settings.json`` está versionado en git
10. Respaldo y recuperación
11. Continuación

Estado: completo · Última revisión: 2026-08-27 · Versión analizada: 0.5.1 (commit f840055)

1. Conclusión previa: no existe motor de base de datos

ChofyAI Studio **no incorpora ningún motor de base de datos**: ni relacional (SQLite, PostgreSQL, MySQL), ni documental (MongoDB), ni clave-valor embebido (sled, RocksDB, Redis), ni ORM/driver alguno. Toda la persistencia se resuelve con **ficheros JSON**, **ficheros YAML**, **ficheros de log en texto plano**, el **árbol de directorios** de `studio_home` y el `localStorage` del WebView.

Este documento trata esos almacenes con el mismo rigor con el que se documentaría un esquema relacional: inventario, diccionario de datos campo a campo, relaciones, cardinalidades, reglas de integridad, concurrencia y política de respaldo.

Documentos relacionados: [05 · Referencia técnica](#), [06 · Explicación profunda del código](#), [08 · Flujo de datos](#) y [11 · Seguridad](#).

1.1. Evidencia de la búsqueda

Las comprobaciones se ejecutaron sobre el árbol del repositorio excluyendo `node_modules/` y `src-tauri/target/`. Comandos y resultados:

```
# 1) Dependencias Rust declaradas
ChofyAI Studio · Q7 · Base de datos y persistencia v0.5.1 (commit f840055) · 2026-08-28
grep -niE "sqlite|postgres|mysql|diesel|sqlx|rusqlite|sea-
orm|mongodb|redis|surreal|duckdb|sled|rocksdb|tauri-plugin-sql" src-tauri/Cargo.toml
# → sin coincidencias

# 2) Dependencias Node declaradas
grep -niE
"sqlite|postgres|mysql|prisma|typeorm|sequelize|knex|drizzle|mongoose|dexie|lowdb|pouchdb|indexeddb|
sql" package.json
# → sin coincidencias

# 3) Uso en código fuente y datos
grep -rniE "sqlite|postgres|mysql|diesel|sqlx|rusqlite|indexedDB|\\bSQL\\b" src src-tauri/src apps
workflows marketplace
# → sin coincidencias

# 4) Artefactos de base de datos en disco
find . -name "*.sql" -o -name "*.db" -o -name "*.sqlite*" -o -name "migrations"
# → sin resultados

# 5) Árbol de dependencias transitivas de Rust
grep -niE '^name = "(rusqlite|sqlx|diesel|libsqlite3-sys|postgres|mysql).*"' src-tauri/Cargo.lock
# → sin coincidencias
```

El bloque [dependencies] completo de `src-tauri/Cargo.toml` es: `serde`, `serde_json`, `serde_yaml`, `thiserror`, `tauri`, `walkdir`. Ninguna aporta acceso a base de datos. Las dependencias de `package.json` son `@tauri-apps/api`, `react` y `react-dom`.

Consecuencias directas de esta decisión: no hay migraciones, no hay transacciones, no hay índices, no hay tipos fuertes en el almacén (la validación vive en `serde` y en el código), y **no hay atomicidad** en las escrituras (ver [sección 8](#)).

2. Inventario de almacenes

#	Almacén	Formato	Ubicación	Escrito por	Ciclo de vida
A1	settings.json	JSON	<repo>/storage/state/settings.json o <app_data_dir>/state/settings.json	Backend Rust	Permanente
A2	processes.json	JSON	Hermano de settings.json	Backend Rust	Volátil-persistente (PIDs)
A3	crash.log	Texto (append)	Hermano de settings.json	Backend Rust (invocado por la UI)	Permanente, crece sin límite
A4	Logs de instalación	Texto	<studio_home>/logs/<tool>-install.log	Backend Rust	Sobrescrito en cada instalación
A5	Logs de ejecución	Texto	<studio_home>/logs/<tool>-run.log	Proceso hijo (stdout/stderr)	Truncado en cada arranque
A6	Logs de descarga de modelos	Texto	<studio_home>/logs/<tool>-model-download.log	Backend Rust	Sobrescrito en cada descarga
A7	Manifests de herramientas	YAML	apps/*.yaml	Repositorio + import_marketplace_tool	Versionado en git
A8	Workflows	YAML	workflows/*.yaml	Repositorio + save_workflow / delete_workflow	Versionado en git
A9	Catálogo de marketplace	YAML	marketplace/registry.yaml	Solo repositorio (sólo lectura en runtime)	Versionado en git
A10	Árbol de studio_home	Sistema de ficheros	<studio_home>/{tools,models,outputs,cache,logs,modules}	Scripts de instalación y herramientas	Permanente
A11	localStorage	Clave-valor del WebView	Perfil del WebView del usuario	Frontend React	Por usuario y por máquina

2.1. Resolución de la ruta de `settings.json`

ChofyAI Studio · 07 · Base de datos y persistencia

v0.5.1 (commit f840055) · 2026-08-28

La función `app_paths()` en `src-tauri/src/system.rs` decide entre dos modos:

```
fn app_paths(app: &AppHandle) -> Result<AppPaths, String> {
    if let Some(root) = repo_root() {
        return Ok(AppPaths {
            apps_dir: root.join("apps"),
            settings_path: root.join("storage").join("state").join("settings.json"),
        });
    }
    let app_data_dir = app.path().app_data_dir().map_err(|e| e.to_string());
    Ok(AppPaths {
        apps_dir: resolve_resource_path(app, "apps")?,
        settings_path: app_data_dir.join("state").join("settings.json"),
    })
}
```

`repo_root()` devuelve `Some` sólo si el directorio de trabajo (o su padre, cuando el `cwd` es `src-tauri`) contiene simultáneamente `apps/`, `scripts/` y `src-tauri/`. Es decir: **en desarrollo el estado se escribe dentro del repositorio** y en la app empaquetada se escribe en el directorio de datos de la aplicación (en macOS, `~/Library/Application Support/cl.vladimiracuna.chofyai.studio/state/`, derivado del identificador de `tauri.conf.json`).

`processes.json` y `crash.log` se derivan siempre del **directorio padre** de `settings_path`, vía `processes_state_path()`, así que los tres almacenes viven juntos.

Observación relevante: `tauri.conf.json` lista `../storage/state/settings.json` entre los `bundle.resources`, de modo que el `settings.json` del repositorio viaja dentro del `.app`, pero el código empaquetado **no lo lee desde ahí** (usa `app_data_dir`). El recurso empaquetado es, en la práctica, inerte.
Inferencia basada en el código.

3. Diccionario de datos

3.1. A1 — `settings.json` (`AppSettings`)

Definido en `src-tauri/src/models.rs` y espejado en `src/types.ts`. Serializado con `serde_json::to_string_pretty`.

Campo	Tipo	Obligatorio	Valor por defecto	Lo escribe	Lo lee	Si falta o es inválido
ChofyAI Studio · 07 · Base de datos y persistencia · v0.5.1 (commit f840039) · 2026-08-28						
studio_home	String	Sí	~/ChofyAIStudio (default_studio_home())	save_studio_home	resolve_effective_home, get_system_summary, list_tools, todos los comandos de tool	Su ausencia hace fallar el parseo del objeto completo; load_settings descarta el fichero entero y devuelve los defaults
tool_overrides	HashMap<String, String>	No (#[serde(default)])	{}	relocate_module (inserta), clear_module_override (elimina)	manifest_install_dir, list_tools (relocated)	Se asume mapa vacío: todas las tools usan su ruta canónica
fallback_home	Option<String>	No	null → ~/ChofyAIStudio	Ningún comando Tauri lo escribe (edición manual)	fallback_home_for dentro de resolve_effective_home	Se usa ~/ChofyAIStudio
sparsebundle_path	Option<String>	No	null	Ningún comando Tauri lo escribe (edición manual)	resolve_effective_home (auto-montaje con hdiutil attach)	Sólo se prueba la convención <studio_home>.sparsebundle
models_dir	Option<String>	No	null → <studio_home>/models	save_path_settings	effective_models_dir, apply_path_env → CHOFYAI_MODELS_DIR	Se usa el default derivado de studio_home
outputs_dir	Option<String>	No	null → <studio_home>/outputs	save_path_settings	effective_outputs_dir, apply_path_env → CHOFYAI_OUTPUTS_DIR	Ídem
cache_dir	Option<String>	No	null → <studio_home>/cache	save_path_settings	effective_cache_dir, apply_path_env → CHOFYAI_CACHE_DIR	Ídem

Notas de integridad:

1. `load_settings` es **deliberadamente tolerante**: envuelve lectura y parseo en `if let Ok(...)` y, ante cualquier error devuelve un `AppSettings` por defecto sin avisar ni reescribir el fichero. Un `settings.json` corrupto se traduce en una app que aparentemente "olvidó" la configuración, con el fichero corrupto intacto en disco.
2. Los tres campos `models_dir` / `outputs_dir` / `cache_dir` se introdujeron después: el `settings.json` versionado en el repositorio sólo contiene cuatro claves y se parsea igual gracias a `#[serde(default)]`.
3. `save_path_settings` normaliza cadenas vacías a `None`, de modo que "vaciar el campo en la UI" equivale a "volver al valor por defecto".
4. `save_studio_home` normaliza cadena vacía a `default_studio_home()` y aplica `trim()`; **no valida** que la ruta sea absoluta, ni que exista, ni que sea escribible. La validación real ocurre después, en `resolve_effective_home`.

3.2. A1b — Mapa `tool_overrides`

Aspecto	Valor
Clave	<code>tool_id</code> — debe coincidir con <code>RawManifest.id</code>
Valor	Ruta del directorio de instalación
Absoluta vs relativa	Si es absoluta se respeta tal cual; si es relativa se resuelve contra <code>studio_home efectivo(manifest_install_dir)</code>
Escritor	<code>relocate_module</code> tras mover físicamente el directorio
Borrado	<code>clear_module_override</code> (quita la entrada sin mover ficheros)
Efecto en la UI	<code>ToolSummary.relocated = tool_overrides.contains_key(id)</code>
Entrada huérfana	Una clave que no corresponde a ningún manifest se ignora silenciosamente; no hay recolección de basura

3.3. A2 — `processes.json` (mapa `tool_id` → PID)

Serialización directa de `HashMap<String, u32>` con `serde_json::to_string_pretty`. No hay struct intermedio.

ChofyAI Studio · 07 · Base de datos y persistencia

Campo	Tipo	Obligatorio	Defecto	Lo escribe	Lo lee	Si falta o es corrupto
clave	String (tool_id)	Sí	—	persist_registry	restore_registry	—
valor	u32 (PID del sistema)	Sí	—	start_tool, restart_tool, adopt_orphan (inserción); stop_tool, health_check_tool (borrado)	list_running_pids, list_orphan_ports, health_check_tool	restore_registry hace return silencioso: el fichero se conserva y la app arranca con el registro en memoria vacío

Ciclo de vida:

- **Arranque:** restore_registry (llamado desde el setup de src-tauri/src/lib.rs) lee el fichero, filtra los PID muertos con pid_is_alive (kill -0) y reescribe el fichero ya depurado.
- **Escritura:** cada mutación del registro llama a persist_registry, que hace fs::create_dir_all del padre y fs::write del JSON completo.
- **Errores ignorados:** persist_registry descarta todos los Result con let _ =. Si el volumen está lleno o de sólo lectura, el estado en memoria y el de disco divergen sin ninguna señal.

3.4. A3 — crash.log

Escrito por append_crash_log (comando Tauri) en modo create(true).append(true).

Campo	Formato	Origen
Marca de tiempo	[<segundos desde epoch UNIX>]	SystemTime::now(); ante error se escribe 0
Mensaje	Texto libre en una línea lógica (puede contener saltos, escritos tal cual)	Parámetro message

El único emisor identificado en el frontend es AppErrorBoundary.componentDidCatch en src/App.tsx, que envía [UI] <mensaje>\n<stack>\n--- componentStack ---\n<componentStack> truncado a 4000 caracteres. La lectura la hace read_crash_log, que devuelve sólo las últimas 200 líneas. El fichero no rota ni se purga: crece indefinidamente.

3.5. A4/A5/A6 — Logs por herramienta

Todos viven en log_dir(studio_home) = <studio_home>/logs/.

ChofyAI Studio · 07 · Base de datos y persistencia

Fichero	Escritor	Modo	Contenido
<tool_id>-install.log	run_install_script	fs::write (sobrescribe) al terminar el proceso	v0.5.1 (commit f840055) · 2026-08-28 stdout completo + "\n" + stderr completo
<tool_id>-run.log	El proceso hijo	fs::File::create (trunca) antes del spawn ; el hijo escribe en vivo	stdout y stderr del servidor de la herramienta, mezclados (try_clone del mismo descriptor)
<tool_id>-model-download.log	download_tool_model	fs::write (sobrescribe) al terminar	Salida del helper scripts/mac/download-hf-model.sh

Consecuencias operativas:

- El log de instalación **sólo existe cuando el proceso termina**. Si la app muere a mitad de una instalación no queda rastro en disco; el streaming en vivo (evento install-progress) se pierde con la ventana.
- El log de ejecución se **trunca en cada start_tool / restart_tool** : no hay histórico entre arranques.
- La lectura desde la UI usa read_tool_log , que acepta kind ∈ {install, run} (cualquier otro valor devuelve error) y devuelve las últimas last_lines líneas (500 por defecto). El log de descarga de modelos **no es legible por read_tool_log** : sólo se puede abrir con open_tool_log ... que tampoco lo contempla, ya que sólo busca -run.log y -install.log . En la práctica, el -model-download.log únicamente es accesible por la ruta absoluta que devuelve ActionResult.log_path .

3.6. A7 — Manifests de herramientas (apps/*.yaml → RawManifest)

RawManifest (privado, en system.rs) es el **esquema real** que el backend lee. Los campos Option<...> son opcionales de facto: serde deserializa su ausencia a None . Los campos no opcionales son obligatorios y su ausencia aborta el parseo.

Campo YAML	Tipo Rust	Obligatorio	Defecto	Lo lee	Si falta / es
ChofyAI Studio · 07 · Base de datos y persistencia				v0.5.1 (commit 1840039) · 2026-08-28	
id	String	Sí	—	find_manifest, manifest_inst, all_dir, tool_override	Error de parseo → collect_manifests devuelve Err → falla la lista completa de tools
name	String	Sí	—	Orden de la lista, mensajes de la UI	Ídem
icon	Option<String>	No	None	ToolSummary.icon; la UI cae al emoji de categoría	Se usa CATEGORY_EMOJI[category] o 🧩
category	String	Sí	—	Agrupación en la UI	Rust no valida el valor; TypeScript lo tipa como unión cerrada y el CI sí lo valida
runtime	String	Sí	—	Sólo informativo en la UI	Rust no valida; CI sí
description	Option<String>	No	"" (unwrap_or_default)	Tarjeta de la tool	Descripción vacía
recommended	Option<bool>	No	false	Badge "recomendada"	Sin badge
default_port	Option<u16>	No	None	health_check_tool, list_orphan_ports, pre-flight de puerto, opened_url, iframe embebido	Sin health check por puerto, sin vista embebida, sin detección de huérfanos
studio_home_subdir	Option<String>	No	tools/<id>	manifest_inst, all_dir	Se usa la convención tools/<id>
install_script	Option<String>	No	None	resolve_install_script (fallback)	Si tampoco hay install_scripts para la plataforma, install_tool devuelve error
install_scripts	Option<HashMap<String, String>>	No (#[serde(default)])	None	resolve_install_script (precedencia)	Se cae a install_script

Campo YAML	Tipo Rust	Obligatorio	Defecto	Lo lee	Si falta / es
ChofyAI Studio · 07 · Base de datos y persistencia				v0.5.1 (commit 1840039) · 2026-08-28	inválido
installed_if	Option<Vec<String>>	No	[]	list_tools , post-validación de instalación, pre- validación de start_tool y update_tool	Con lista vacía, installed es siempre false (ver 3.6.2)
run.command	Option<String>	No	None	resolve_run_command (fallback)	start_tool / restart_tool fallan
run.commands	Option<HashMap<String,String>>	No (#[serde(default)])	None	resolve_run_command (precedencia)	Se cae a run.command
platforms	Option<Vec<String>>	No	None ⇒ sólo mac-arm64	platform_supported	Ausente equivale a declarar únicamente mac- arm64 (retrocompatibilidad)
models	Option<Vec<String>>	No	[]	list_declared_models , download_tool_model	La tool no ofrece descargas de modelos declarados

3.6.1. Campos declarados en los YAML que el backend NO lee

RawManifest no declara #[serde(deny_unknown_fields)] , así que **cualquier clave desconocida se ignora en silencio**. Confirmado contra el struct, estos campos aparecen en los manifests del repositorio pero no tienen efecto en runtime:

Campo	Presente en	¿Lo lee RawManifest ?	Consumidor real
python_manager	aceforge.yaml , comfyui.yaml , facefusion.yaml , qwen3-tts.yaml	No	No identificado en el código Rust ni en el frontend; documentación de intención (auto \
healthcheck	qwen3-tts.yaml	No	El health check real es health_check_tool , que usa default_port y un TcpStream::connect_timeout ; el bloque type/url es decorativo
install	qwen3-tts.yaml (lista de comandos git/venv)	No	La instalación real la hace scripts/mac/install-qwen3-tts.sh
notes	qwen3-tts.yaml	No	Sólo lectura humana

3.6.2. Semántica exacta de `installed`

En `list_tools`:

```
installed: missing_checks.is_empty() && !installed_checks.is_empty(),
```

Es decir: una tool se considera instalada **sólo si declara al menos un `installed_if` y todos los artefactos existen** bajo el directorio de instalación resuelto. Un manifest sin `installed_if` aparecerá siempre como "no instalada", aunque el directorio esté lleno. Es una regla de integridad deliberada: sin criterio declarado, no hay verificación posible.

3.7. A8 — Workflows (`workflows/*.yaml`)

El backend **no tiene un struct tipado** para los workflows: `list_workflows` parsea a `serde_yaml::Value` y lo reemite como `serde_json::Value`. El esquema efectivo lo define TypeScript en `src/types.ts`.

WorkflowDef :

Campo	Tipo TS	Obligatorio	Validado por	Consumidor
<code>id</code>	<code>string</code>	Sí	<code>save_workflow</code> (presencia) y <code>validate_workflo</code> <code>w_id</code> (formato del nombre de fichero)	Orden en <code>list_workflows</code> , nombre del <code>.yaml</code>
<code>name</code>	<code>string</code>	Sí	<code>save_workflow</code> (presencia)	Cabecera del <code>WorkflowRunner</code>
<code>category</code>	<code>string</code>	Sí en el tipo TS, no validado por Rust	—	Agrupación en el panel
<code>emoji</code>	<code>string?</code>	No	—	Icono; cae a 
<code>description</code>	<code>string</code>	Sí	<code>save_workflow</code> (presencia)	Texto del modal
<code>requires_tools</code>	<code>string[]?</code>	No	Nadie lo valida ni lo aplica	Informativo en la UI
<code>inputs</code>	<code>WorkflowInput[]?</code>	No	—	Formulario del runner
<code>steps</code>	<code>WorkflowStep[]</code>	Sí	<code>save_workflow</code> (presencia)	Bucle de ejecución

WorkflowInput :

ChofyAI Studio · 07 · Base de datos y persistencia

Campo	Tipo	Obligatorio	Defecto	Efecto
id	string	Sí	—	Clave en <code>inputs / files</code> y en <code>{{inputs.<id>}}</code>
type	<code>'file' \</code>	<code>'text'</code>	Sí	—
label	string	Sí	—	Etiqueta visible
required	boolean?	No	false	Alimenta <code>canRun</code>
default	string?	No	—	Valor inicial del estado <code>inputs</code>
accept	string?	No	—	Atributo <code>accept</code> del input de fichero
placeholder	string?	No	—	Placeholder del input de texto

WorkflowStep :

Campo	Tipo	Obligatorio	Efecto
id	string	Sí	Clave de <code>StepResult</code>
label	string	Sí	Texto del paso
type	<code>'http' \</code>	<code>'stub'</code>	Sí
method	<code>'GET' \</code>	<code>'POST'?</code>	No
url	string?	Requerido si <code>type: http</code>	Destino del <code>fetch</code> ; admite <code>{{inputs.x}}</code>
body_kind	<code>'multipart' \</code>	<code>'json'?</code>	No
fields	<code>Record<string,string> ?</code>	No	Campos del <code>FormData</code> ; el marcador <code>__FILE__:<id></code> inyecta el <code>File</code> real
body	string?	No	Cuerpo JSON en crudo, con sustitución de variables
note	string?	No	Texto devuelto por los pasos <code>stub</code>
input_from	string?	No	Inyecta la salida del paso anterior bajo esa clave
output	<code>{ kind, from?, label? }?</code>	No	<code>from</code> extrae una propiedad del JSON de respuesta

Los tres workflows del repositorio (`transcribe-audio` , `comfyui-prompt` , `audio-pipeline`) respetan este esquema. `audio-pipeline` contiene dos pasos `stub` declarados como tales.

3.8. A9 — marketplace/registry.yaml (MarketplaceEntry)

ChofyAI Studio · 07 · Base de datos y persistencia

v0.5.1 (commit f840055) · 2026-08-28

Fichero con una única clave raíz `tools:` (struct `MarketplaceFile`). El backend lo lee desde `<repo>/marketplace/registry.yaml` o, en modo empaquetado, desde el recurso `marketplace/registry.yaml`. Es **sólo lectura**: ningún comando lo escribe.

Campo	Tipo Rust	Obligatorio	Uso en <code>import_marketplace_tool</code>
<code>id</code>	<code>String</code>	Sí	Nombre del fichero <code>apps/<id>.yaml</code> y campo <code>id</code>
<code>name</code>	<code>String</code>	Sí	Campo <code>name</code>
<code>category</code>	<code>String</code>	Sí	Campo <code>category</code>
<code>runtime</code>	<code>String</code>	Sí	Campo <code>runtime</code>
<code>short_description</code>	<code>String</code>	Sí	Campo <code>description</code> , con <code>\n</code> reemplazados por espacios
<code>homepage</code>	<code>Option<String></code>	No	Se devuelve en <code>ActionResult.opened_url</code>
<code>repo</code>	<code>Option<String></code>	No	Comentario <code># Repo:</code> (se omite si es cadena vacía)
<code>default_port</code>	<code>Option<u16></code>	No	Campo <code>default_port</code> si está presente
<code>estimated_size_gb</code>	<code>Option<u32></code>	No	Sólo informativo en la UI
<code>requires</code>	<code>Option<Vec<String>></code>	No	Sólo informativo en la UI
<code>install_hint</code>	<code>Option<String></code>	No	Comentario <code># Hint de instalación:</code>
<code>notes</code>	<code>Option<String></code>	No	Comentario <code># Notas del marketplace:</code>

El manifest generado siempre fija `recommended: false`, `studio_home_subdir: tools/<id>`, `platforms: [mac-arm64]` e `installed_if: [source/.git]`, sin `install_script` ni `run`: la tool importada aparece en la lista pero no es instalable hasta que alguien edite el YAML a mano.

3.9. A10 — Árbol de `studio_home`

`studio_home` es el "tablespace" del sistema: el volumen donde vive todo lo pesado.

ChofyAI Studio · 07 · Base de datos y persistencia

Subdirectorio	Creado por	Contenido	Quién lo lee
tools/<id>/	Scripts de instalación (mkdir -p "\$INSTALL_DIR")	Clon del repositorio, venv, binarios compilados	v0.5.1 (commit f840055) · 2026-08-28 installed_if , start_tool (current_dir), open_tool_directory
tools/<id>/models/	resolve_models_dir + scripts	Modelos por herramienta	list_tool_models , list_declared_models , delete_tool_model , download_tool_model
models/	install-qwen3-tts.sh , o el override models_dir	Modelos compartidos	Los scripts, vía CHOFYAI_MODELS_DIR
outputs/	Scripts de instalación	Salidas generadas por las herramientas	Los scripts, vía CHOFYAI_OUTPUTS_DIR
cache/	Scripts de instalación	Caché de descargas (p. ej. HF_HOME)	Los scripts, vía CHOFYAI_CACHE_DIR
logs/	fs::create_dir_all(&logs) en Rust	A4, A5, A6	read_tool_log , open_tool_log
modules/	relocate_module , indirectamente	Herramientas reubicadas	Vía tool_overrides
.chofyai-write-probe	is_writable_dir	Fichero temporal de cero bytes	Se borra inmediatamente; si el proceso muere entre write y remove_file , queda residual

Sobre modules/ : no existe código que cree ese directorio por sí mismo. El frontend propone `${studio_home_effective}/modules/${tool.id}` como destino por defecto en `startRelocate` , y es `relocate_module` quien crea el padre con `fs::create_dir_all` . Es una convención de la UI, no una estructura garantizada.

`resolve_models_dir` merece una aclaración importante: **ignora el override `settings.models_dir`** . Devuelve siempre `manifest_install_dir(...)/models` , es decir `<studio_home>/tools/<id>/models` . El override `models_dir` sólo llega a los scripts como variable de entorno `CHOFYAI_MODELS_DIR` . Los comandos de gestión de modelos de la UI y los scripts pueden, por tanto, apuntar a directorios distintos. *Inferencia basada en el código; requiere validación funcional.*

3.10. A11 — localStorage del frontend

ChofyAI Studio · 07 · Base de datos y persistencia

v0.5.1 (commit f840055) · 2026-08-28

Clave	Tipo	Valores	Defecto si falta o falla	Escritor	Lector
chofyai_onboarding_done	string	'1'	Ausente ⇒ se muestra el onboarding	Onboarding.finish en src/App.tsx	Inicializador de showOnboarding
chofyai_theme	string	'dark', 'light', 'system'	'dark'	Efecto de theme en src/App.tsx	Inicializador de theme
chofyai_lang	string	'es', 'en' (ver src/i18n.ts)	Detección por navegador y, en último término, 'es'	setLang en src/i18n.ts	getLang en src/i18n.ts

Los tres accesos están envueltos en `try/catch` con fallback silencioso, así que un WebView con almacenamiento deshabilitado degrada correctamente. Ningún valor de `localStorage` viaja al backend ni se sincroniza entre máquinas.

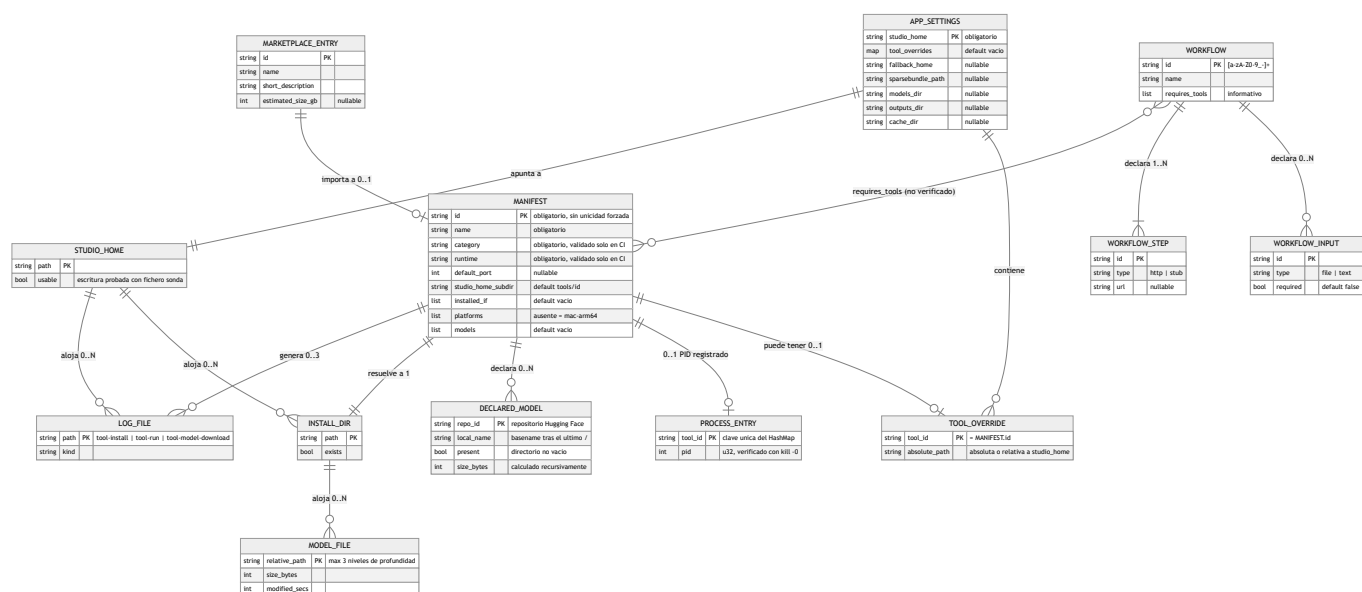
4. Relaciones entre entidades

Las relaciones no las impone ningún motor: las materializa el código, siempre por **coincidencia de cadenas** (`tool_id`) o por **convención de rutas**.

Origen	Destino	Clave	Cardinalidad	Dónde se resuelve
ChofyAI Studio · 07 · Base de datos y persistencia AppSettings.tool_overrides	RawManifest.id	tool_id	0..1 override por manifest	v0.5.1 (commit f840055) · 2026-08-28 manifest_install_dir
processes.json	RawManifest.id	tool_id	Como máximo un PID por manifest (es un HashMap, la clave es única)	ProcessRegistry
RawManifest	Modelos declarados	models[] (repo de Hugging Face)	1 manifest → 0..N modelos	list_declared_models
RawManifest	Ficheros de log	tool_id en el nombre del fichero	1 manifest → 0..3 logs	log_dir + formato <tool>-<kind>.log
RawManifest	Directorio de instalación	studio_home_subdir o tools/<id>	1 → 1	manifest_install_dir
WorkflowDef.requires_tools[]	RawManifest.id	tool_id	N a N, declarativa y no verificada	Sólo se muestra en la UI
WorkflowStep.url	Puerto de la herramienta	Puerto literal en la URL	Acoplamiento por convención	runWorkflowStep
MarketplaceEntry.id	apps/<id>.yaml	id	1 → 0..1	import_marketplace_tool

El acoplamiento por puerto literal es frágil: `transcribe-audio.yaml` codifica `http://127.0.0.1:8178/inference` y `comfyui-prompt.yaml` codifica `http://127.0.0.1:8188/prompt`. Si alguien cambia `default_port` en el manifest, los workflows no se enteran.

4.1. Diagrama entidad-relación



Lectura del diagrama, punto por punto:

- **MANIFEST** `||--o|` **PROCESS_ENTRY** : la cardinalidad máxima de 1 no es una restricción declarada, sino una consecuencia de usar `HashMap<String, u32>`. Si un `start_tool` se ejecuta dos veces sin `stop_tool` intermedio, la segunda inserción **sobrescribe** el PID anterior y el primer proceso queda huérfano: sigue vivo, ocupa el puerto y sólo `list_orphan_ports` lo encuentra.
- **MANIFEST** `||--o{` **DECLARED_MODEL** : sólo `qwen3-tts.yaml` declara `models` hoy (tres repositorios `mlx-community/*`).
- **WORKFLOW** `}o--o{` **MANIFEST** : relación puramente documental. Ejecutar un workflow cuyo `requires_tools` no esté instalado produce un error de red en el paso HTTP, no una validación previa.
- **MARKETPLACE_ENTRY** `||--o|` **MANIFEST** : la importación es unidireccional y no deja traza del origen más allá de comentarios YAML.

5. Reglas de integridad implementadas

ChofyAI Studio · 07 · Base de datos y persistencia

v0.5.1 (commit f840055) · 2026-08-28

Regla	Dónde	Qué garantiza	Qué NO garantiza
<code>installed_if</code> como criterio de instalación	<code>list_tools</code> , <code>run_install_script</code> (post-validación), <code>start_tool</code> (pre-validación), <code>update_tool</code>	Que no se arranque una instalación parcial ni se declare instalada una tool incompleta	Que los artefactos sean funcionales; sólo comprueba existencia de rutas
Post-validación tras instalación exitosa	<code>run_install_script</code>	Que un script que devuelve código 0 pero dejó el <code>venv</code> a medias se reporte como error	Nada sobre integridad de contenidos
<code>validate_workflow_id</code>	<code>save_workflow</code> , <code>delete_workflow</code>	<code>id</code> no vacío, sin <code>/</code> , <code>\</code> ni <code>..</code> , y limitado a <code>[a-zA-Z0-9_-]</code>	Que el <code>id</code> del YAML coincida con el nombre del fichero
Campos obligatorios del workflow	<code>save_workflow</code>	Presencia de <code>id</code> , <code>name</code> , <code>description</code> , <code>steps</code> en el mapping raíz	Tipos, forma de <code>steps</code> , validez de las URLs
Guarda de path traversal	<code>delete_tool_model</code>	Doble comprobación: rechazo léxico de <code>..</code> y <code>canonicalize() + starts_with(canonical_root)</code> ; además exige que el objetivo sea un fichero	Nada sobre enlaces simbólicos creados <i>después</i> de la canonicalización (ventana TOCTOU teórica)
Modelo declarado antes de descargar	<code>download_tool_model</code>	Que sólo se descarguen repositorios listados en <code>manifest.models</code> ; impide inyectar un <code>repo_id</code> arbitrario desde la UI	Que el repositorio exista en Hugging Face
No sobrescritura de manifests	<code>import_marketplace_tool</code>	Que <code>apps/<id>.yaml</code> existente aborte la importación con error	Unicidad del campo <code>id</code> dentro de los YAML
Plataforma soportada	<code>platform_supported</code> en <code>run_install_script</code>	Que no se lance un script de instalación en una plataforma no declarada	Nada en <code>start_tool</code> : arrancar no revalida la plataforma
Pre-flight de puerto	<code>start_tool</code>	Mata con <code>kill -9</code> a cualquier proceso ajeno que escuche en <code>default_port</code>	Es una medida agresiva: puede matar procesos legítimos del usuario que compartan puerto
Destino vacío al reubicar	<code>relocate_module</code>	Ruta absoluta obligatoria, destino distinto del origen, destino inexistente o vacío, padre escribible	Atomicidad del movimiento (ver sección 8)
Validación de manifests en CI	<code>validate-manifests</code> en <code>.github/workflows/ci.yaml</code>	Presencia de <code>id</code> , <code>name</code> , <code>category</code> , <code>runtime</code> , <code>description</code> , <code>platforms</code> ,	Nada en runtime: un YAML editado a mano tras la instalación no pasa por el CI

Regla	Dónde	Qué garantiza	Qué NO garantiza
ChofyAI Studio · 07 · Base de datos y persistencia		<code>installed_if</code> ; regla condicional <code>install_script</code> ⇒ <code>run</code> ; valores válidos de <code>category</code> y <code>runtime</code>	v0.5.1 (commit f840055) · 2026-08-28

5.1. Unicidad del `id` de manifest

No está forzada en runtime. `collect_manifests` recorre `apps/*.yaml` (profundidad exactamente 1) y ordena por `name`; `find_manifest` devuelve el **primer** manifest cuyo `id` coincide. Si dos ficheros declararan el mismo `id`, el que gane dependería del orden alfabético de `name` y el otro quedaría inaccesible sin ningún aviso. La única protección real es indirecta: `import_marketplace_tool` nombra el fichero como `<id>.yaml` y rechaza sobrescribir, lo que hace improbable la colisión por la vía de importación, pero no por edición manual. El CI tampoco comprueba duplicados.

6. Modo de fallo por almacén

Escenario	Efecto observado	Recuperación
<code>settings.json</code> corrupto o ilegible	<code>load_settings</code> devuelve defaults; el <code>studio_home</code> pasa a <code>~/ChofyAISTudio</code> ; las tools instaladas en el volumen externo aparecen como no instaladas	Manual: reparar o borrar el fichero
<code>settings.json</code> inexistente	Igual que el anterior; se recrea en el primer <code>save_*</code>	Automática al guardar
<code>processes.json</code> corrupto	<code>restore_registry</code> retorna sin tocar el registro; los procesos vivos quedan como huérfanos	<code>list_orphan_ports</code> + <code>adopt_orphan</code>
PID reciclado por el sistema operativo	<code>pid_is_alive</code> da <code>true</code> para un proceso ajeno; <code>stop_tool</code> enviaría <code>SIGTERM</code> a un proceso que no es nuestro	No hay mitigación en el código
Un solo <code>apps/*.yaml</code> malformado	<code>collect_manifests</code> propaga el <code>Err: list_tools falla y la UI cae a fallbackTools</code> (las cinco tools hardcodeadas en <code>src/App.tsx</code>)	Corregir el YAML
<code>marketplace/registry.yaml</code> malformado	<code>list_marketplace_tools</code> devuelve <code>Err</code> ; el panel muestra el error vía toast	Corregir el YAML
Un <code>workflows/*.yaml</code> malformado	<code>list_workflows</code> lo salta en silencio (<code>if let Ok(v)</code>)	El workflow simplemente no aparece
Log truncado o borrado en caliente	<code>read_tool_log</code> devuelve lo que haya; <code>open_tool_log</code> puede fallar con "No hay logs disponibles"	Reinstalar o rearrancar la tool
<code>studio_home</code> desmontado	<code>resolve_effective_home</code> intenta <code>hdiutil attach</code> del sparsebundle y, si falla, cae a <code>fallback_home</code> ; <code>using_fallback</code> se marca <code>true</code> en <code>SystemSummary</code>	Remontar el volumen y refrescar
Disco lleno al persistir	<code>persist_registry</code> ignora el error; <code>save_settings_to_disk</code> sí devuelve <code>Err</code> y la UI muestra un toast	Liberar espacio

7. Mapa de módulo/función por almacén

Almacén	Funciones que escriben	Funciones que leen
settings.json	save_settings_to_disk ← save_studio_home, save_path_settings, relocate_module, clear_module_override	load_settings ← get_system_summary, get_effective_paths, list_tools, install_tool, update_tool, start_tool, restart_tool, open_tool_directory, open_tool_log, read_tool_log, get_system_stats, resolve_models_dir, download_tool_model
processes.json	persist_registry ← start_tool, restart_tool, stop_tool, adopt_orphan, health_check_tool, restore_registry	restore_registry
crash.log	append_crash_log ← AppErrorBoundary.componentDidCatch	read_crash_log
<tool>-install.log	run_install_script	read_tool_log, open_tool_log
<tool>-run.log	Proceso hijo (descriptor creado en start_tool / restart_tool)	read_tool_log, open_tool_log
<tool>-model-download.log	download_tool_model	Sólo por ruta absoluta (ActionResult.log_path)
apps/*.yaml	import_marketplace_tool	collect_manifests ← find_manifest, list_tools, list_orphan_ports
workflows/*.yaml	save_workflow, delete_workflow	list_workflows
marketplace/registry.yaml	— (sólo lectura)	list_marketplace_tools
Árbol de studio_home	resolve_effective_home (create_dir_all), run_install_script (logs/), start_tool (logs/), list_declared_models (models/), relocate_module (mover/copiar), delete_tool_model (borrar), open_tool_directory (create_dir_all), scripts de scripts/mac/	list_tools (installed_if), list_tool_models, dir_size, read_disk_usage

Almacén	Funciones que escriben	Funciones que leen
ChofyAI Studio · 07 · Base de datos y persistencia localStorage	Onboarding.finish , efecto de theme , setLang	v0.5.1 (commit f840055) · 2026-08-28 Inicializadores de showOnboarding y theme , getLang

8. Concurrency y transaccionalidad

8.1. Lo que sí está protegido

El único mecanismo de exclusión mutua del sistema es el `Mutex` del registro de procesos, declarado en `system.rs` y gestionado por Tauri en `lib.rs`:

```
pub struct ProcessRegistry(pub Mutex<HashMap<String, u32>>);
```

```
.manage(ProcessRegistry(Mutex::new(HashMap::new())))
```

Todos los accesos hacen `registry.0.lock()` y propagan el envenenamiento del mutex como `Err(String)` mediante `map_err(|e| e.to_string())`. Esto serializa las mutaciones del mapa **dentro del proceso de la aplicación**.

Un detalle importante: en `start_tool`, `restart_tool` y `adopt_orphan`, la llamada a `persist_registry` se hace **con el guard todavía tomado**, dentro del mismo bloque. Eso impide que dos escrituras concurrentes del fichero se intercalen desde hilos distintos del mismo proceso. En `stop_tool` ocurre lo mismo (el `MutexGuard pids` sigue vivo cuando se llama a `persist_registry`).

8.2. Lo que NO está protegido

- Escritura no atómica de `settings.json`.** `save_settings_to_disk` hace `fs::write` directo sobre el fichero de destino: trunca y reescribe. No hay patrón *write-temp + fsync + rename*. Si el proceso muere, el volumen se desmonta o el disco se llena a mitad de la escritura, el fichero queda truncado o vacío. En el siguiente arranque, `load_settings` no distingue "corrupto" de "inexistente" y **silenciosamente vuelve a los valores por defecto**, incluido `studio_home = ~/ChofyAIStudio`. El usuario percibe una pérdida total de configuración.
- Escritura no atómica de `processes.json`.** Mismo patrón, con el agravante de que todos los errores se descartan con `let _ =`.
- Nada coordina múltiples instancias de la app.** No hay fichero de bloqueo, ni `flock`, ni comprobación de instancia única. Dos procesos de ChofyAI Studio apuntando al mismo `settings.json` se pisan mutuamente: el último en escribir gana y el registro de PIDs de uno borra el del otro.
- `apps/*.yaml`, `workflows/*.yaml` y `marketplace/registry.yaml` no tienen bloqueo alguno.** `save_workflow` sobrescribe sin comprobar si el fichero cambió desde que se leyó (no hay control de versiones optimista).
- `relocate_module` no es transaccional.** La secuencia es: intentar `fs::rename`; si falla (típicamente por ser *cross-device*), `copy_dir_recursive` seguido de `fs::remove_dir_all` del origen, y sólo después se persiste el override en `settings.json`. Una interrupción entre la copia y el borrado deja dos copias; entre

6. **La escritura de los logs de ejecución es del proceso hijo**, fuera de todo control: dos herramientas con el mismo `tool_id` escribirían en el mismo fichero.
7. `append_crash_log` usa `O_APPEND`, lo que en POSIX hace atómicas las escrituras pequeñas frente a otros escritores; es el único almacén con una garantía de concurrencia razonable, y es incidental.

8.3. Resumen de garantías

Propiedad ACID	Estado
Atomicidad	Ausente en JSON y en el movimiento de módulos; incidental (<code>O_APPEND</code>) en <code>crash.log</code>
Consistencia	Parcial: la impone el código (<code>installed_if</code> , validaciones), no el almacén
Aislamiento	Sólo intra-proceso, y sólo para el registro de PIDs
Durabilidad	Sin <code>fsync</code> explícito: los datos quedan en la caché del sistema de ficheros hasta que el SO decida

9. Datos personales y sensibles almacenados

Dato	Dónde queda	Sensibilidad	Observaciones
Rutas absolutas del sistema de ficheros	<code>settings.json</code> , logs, <code>crash.log</code> , <code>ToolSummary.install_dir</code> , <code>SystemStats.disk_path</code>	Media: contienen el nombre de usuario del sistema (<code>/Users/<usuario>/...</code>)	Se muestran tal cual en la UI
Nombres de volúmenes externos	<code>settings.json</code> , <code>VolumeCandidate.label</code> , salida de <code>list_external_volumes</code>	Baja-media: revelan hardware conectado	<code>list_external_volumes</code> enumera todo <code>/Volumes</code> , no sólo los del usuario
Contenido de los logs de instalación	<code><tool>-install.log</code>	Media: incluye URLs de repositorios, paquetes Python instalados, salidas de <code>pip</code> / <code>uv</code> , mensajes de error con rutas	Se sobrescriben en cada instalación
Contenido de los logs de ejecución	<code><tool>-run.log</code>	Alta según la herramienta: un servidor de ASR o TTS puede volcar nombres de fichero de audio, prompts o transcripciones	ChofyAI Studio no controla qué imprime cada herramienta
Trazas de error de la UI	<code>crash.log</code>	Media: <i>stack traces</i> y <code>componentStack</code> de React, truncados a 4000 caracteres	Sin rotación: crece sin límite
Preferencias de interfaz	<code>localStorage</code>	Baja	Local al WebView
Credenciales	—	—	No se almacena ninguna credencial: no hay tokens, contraseñas ni claves de API en ningún almacén. El helper <code>download-hf-model.sh</code> delega en <code>huggingface-cli</code> / <code>huggingface_hub</code> , que gestionan su propio token fuera del árbol de la aplicación

9.1. Hallazgo: `settings.json` está versionado en git

`storage/state/settings.json` **está bajo control de versiones** (aparece en `git ls-files storage/`). El `.gitignore` excluye `storage/state/settings.local.json` y `storage/state/runtime/` , pero no el `settings.json` principal. En consecuencia, cada commit que lo toque publica el `studio_home` , el `fallback_home` y la ruta del `.sparsebundle` reales de quien haga el commit: rutas de volúmenes y, potencialmente, el nombre de usuario del sistema. Ver [11 · Seguridad](#) para el tratamiento del riesgo.

10. Respaldo y recuperación

ChofyAI Studio · 07 · Base de datos y persistencia

v0.5.1 (commit f840055) · 2026-08-28

No existe ninguna política de respaldo, ni automática ni asistida. Verificado por ausencia: no hay comandos Tauri de exportación/importación de configuración, no hay copias `.bak` ni versionado de `settings.json`, no hay rotación de logs, no hay snapshots del árbol de `studio_home` ni integración con Time Machine.

Lo más cercano a una recuperación son estos mecanismos, todos parciales:

1. **Auto-depuración de `processes.json`** en `restore_registry`: descarta los PID muertos al arrancar.
2. **Fallback de `studio_home`**: `resolve_effective_home` mantiene la app usable cuando el volumen principal no está, a costa de perder de vista lo instalado allí.
3. **Auto-montaje del sparsebundle** con `hdiutil attach -nobrowse -noverify`.
4. **`run_doctor`**, que ejecuta `scripts/mac/doctor.sh` y devuelve un diagnóstico textual (`stdout` + `stderr`), sin acción correctiva automática.
5. **Reinstalación**: la vía real de recuperación de una herramienta rota es volver a ejecutar su `install_script`.

Recomendación operativa mínima, dado que el estado crítico es diminuto: copiar periódicamente el directorio que contiene `settings.json` y `processes.json`. El contenido pesado (`tools/`, `models/`) es reconstruible desde los scripts y desde la red, salvo `outputs/`, que contiene el trabajo generado por el usuario y **no tiene ningún mecanismo de protección**.

11. Continuación

- El recorrido completo del dato —origen, validación, transformación, destino y salida hacia servicios externos— está en [08 · Flujo de datos](#).
- Las firmas exactas de los 35 comandos Tauri que tocan estos almacenes están en [05 · Referencia técnica](#).
- El razonamiento línea a línea de `resolve_effective_home`, `run_install_script` y el registro de procesos está en [06 · Explicación profunda del código](#).
- El análisis de riesgo de la escritura no atómica, del `kill -9` en el pre-flight de puerto y del `settings.json` versionado está en [11 · Seguridad](#).